

Contents

Contents	2
I. Los Archivos de Asgard	3
II. Prólogo	4
III. Introducción	5
A. Ejercicio 0: El Sello de Odín	6
B. Ejercicio 1: Abriendo la Bóveda	7
C. Ejercicio 2: La Runa Olvidada (<i>serialVersionUID</i>)	8
D. Ejercicio 3: Reliquias en Cadena (Lista Serializable)	9
E. Ejercicio 4: El Traidor (transient)	10
F. Ejercicio 5: El Pergamino Universal (JSON equivalente)	11
G. Ejercicio 6: El Veredicto de Heimdall (comparativa)	12
H. Ejercicio 7: La Línea de Sucesión (herencia + serialización)	13
Reto. "El Vigía del Bifröst"	14
Ejecución	16

I. Los Archivos de Asgard

Ya conoces cómo cerrar recursos de forma segura (***try-with-resources***, **módulo 05**) y ya usaste JSON de pasada (módulo 04). Ahora vas a entender la otra cara de guardar información en Java: la serialización binaria nativa — la runa grabada en piedra que solo un asgardiano (o, en este caso, solo Java) puede leer.

Vas a construir tus propias reliquias serializables, verlas sobrevivir un viaje a disco y regresar intactas, y al final vas a enfrentar cara a cara ambos formatos — binario y JSON — para entender con criterio propio por qué el segundo ganó la batalla en la práctica.



Instrucciones Generales

- ✓ **Versión:** Java 17+ (OpenJDK).
- ✓ **Estructura:** Una clase por archivo (Puedes usar package).
- ✓ **Linter Obligatorio:** Checkstyle (Google Java Style). Errores de estilo = KO.

- ✓ **Comentarios:** Obligatorios, solo en inglés y siguiendo formato Javadoc.
- ✓ **Nomenclatura:** Clases en PascalCase, métodos y variables en camelCase.
- ✓ **IDE:** IntelliJ IDEA (recomendado), VS Code, Eclipse, etc.
- ✓ **Compilación:** Vía terminal: `javac Clase.java` y `java Clase`.

Entrega y Evaluación

- Entrega en repositorio Git.

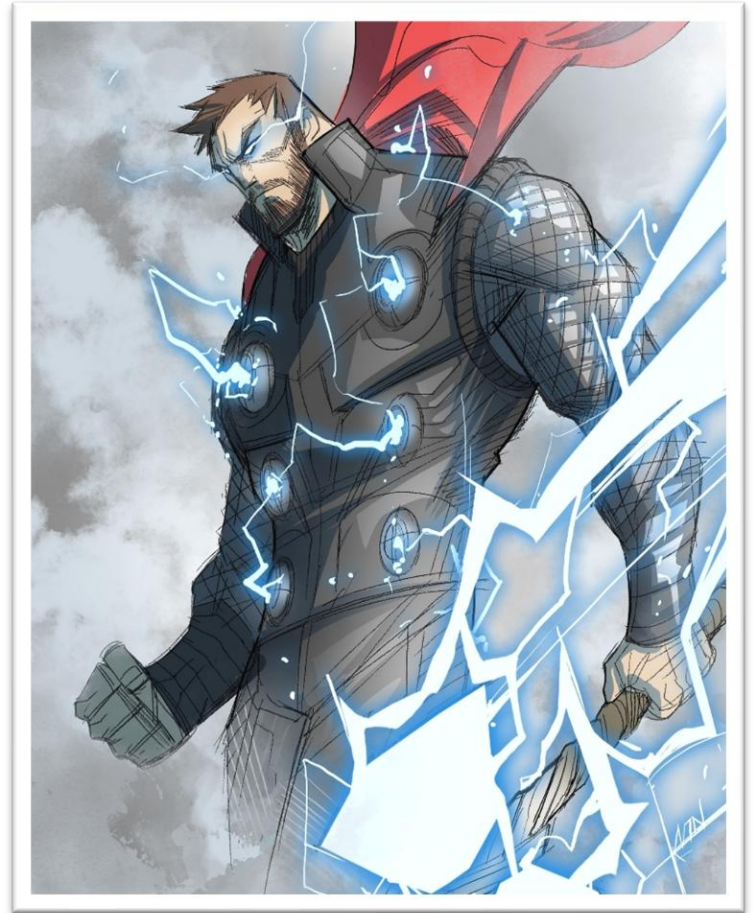
II. Prólogo

En las **bóvedas de Asgard** se guardan reliquias que deben sobrevivir milenios: el Mjolnir, los tratados con los Nueve Reinos, los registros de cada batalla desde la guerra contra los Gigantes de Hielo.

Odín no puede permitirse perder esa información solo porque un Bifröst se desestabilizó a mitad de una transmisión.

Guardar información no es solo escribirla en un papel — es decidir en qué formato sobrevive el tiempo.

Los antiguos escribas asgardianos usaban runas grabadas en piedra (rápidas de crear, pero solo un asgardiano puede leerlas).



Los diplomáticos modernos prefieren pergaminos en el idioma común de los Nueve Reinos (más lentos de escribir, pero cualquier reino los puede leer, incluso Midgard).

En este módulo vas a aprender ambas formas de "grabar en piedra" tus datos: la serialización binaria de Java (rápida, pero solo Java la entiende) y el JSON que ya conoces (más verboso, pero universal). Y vas a entender, con ejemplos reales, por qué en la práctica el segundo ganó la batalla casi siempre.

III. Introducción

¡Bienvenido al Módulo 06: Los Archivos de Asgard!

Ya sabes leer y escribir **JSON (módulo 04)**. Ahora vas a conocer la otra cara de la persistencia de datos en Java: la serialización binaria nativa. Al final del módulo vas a poder comparar ambas con criterio propio, no de memoria.

Aprenderás:

- Qué es Serializable y cómo convertir un objeto en una secuencia de bytes
- Cómo recuperar (deserializar) un objeto exactamente como estaba
- Qué es serialVersionUID y por qué ignorarlo te puede explotar en la cara meses después
- Los riesgos de seguridad de la serialización binaria (por qué muchas empresas la prohíben)
- Comparar de forma práctica: mismo dato, guardado en .ser y en .json — ¿cuál pesa más? ¿cuál es más legible? ¿cuál sobrevive a un cambio de versión de Java? cosas se vuelvan... verdes.



A. Ejercicio 0: El Sello de Odín

- ✓ **Directorio:** ex0/
- ✓ **Archivo:** AsgardianRelic.java, SealTest.java
- ✓ **Funciones Autorizadas:** java.io.Serializable, ObjectOutputStream, FileOutputStream, try/catch.
- ✓ **Método:** Any.
- ✓ **Restricción:**
 - La clase AsgardianRelic debe implementar explícitamente la interfaz Serializable.

Objetivo

En Asgard, las reliquias más poderosas no se almacenan en cofres comunes. Antes de abandonar el reino, deben ser **selladas** para preservar su estado y poder recuperarlas exactamente como eran. En este ejercicio aprenderás el primer paso de la serialización en Java: **guardar un objeto en un archivo**.

Debes crear una clase **AsgardianRelic** con los siguientes atributos:

- **name (String)**
- **powerLevel (int)**

Después, crea una instancia de la reliquia y utiliza un **ObjectOutputStream** junto con un **FileOutputStream** para serializarla en un archivo llamado: **relic.ser**

En este ejercicio **no debes leer el archivo**. El único objetivo es demostrar que el objeto puede serializarse correctamente y que el archivo se genera.

El programa debe:

- Crear la clase **AsgardianRelic**.
- Implementar la interfaz Serializable.
- Crear una instancia de la reliquia. (Ponerle cualquier dato)
- Serializar el objeto en relic.ser.
- Mostrar un mensaje confirmando que el sellado se ha realizado correctamente.

Salida Esperada – relic.ser

```
The relic's seal has been successfully applied!  
Relic sealed into relic.ser
```

B. Ejercicio 1: Abriendo la Bóveda

- ✓ **Directorio:** ex1/
- ✓ **Archivo:** VaultReader.java, relic.ser
- ✓ **Método:** Any.
- ✓ **Funciones Autorizadas:** ObjectInputStream, FileInputStream, Casting
- ✓ **Restricción:**
 - Debes convertir el resultado de **objIn.readObject()** al tipo *AsgardianRelic* mediante un casting explícito.

Objetivo

En el ejercicio anterior sellaste una reliquia de Asgard y la almacenaste en la bóveda. Ahora ha llegado el momento de comprobar que el proceso de serialización ha funcionado correctamente. Tu misión es abrir el archivo relic.ser, recuperar el objeto almacenado y demostrar que conserva exactamente el mismo estado que tenía antes de ser sellado.

Para ello deberás utilizar un **ObjectInputStream** junto con un **FileInputStream** y convertir el resultado de **readObject()** al tipo **AsgardianRelic**.

El programa debe:

- Abrir el archivo relic.ser.
- Leer el objeto almacenado mediante readObject().
- Realizar el *casting* a AsgardianRelic.
- Mostrar los valores de name y powerLevel.
- Demostrar que los datos recuperados coinciden con los que fueron serializados en el ejercicio anterior.

Salida Esperada

```
Relic recovered: XXXXX, power level: XXXXXX
```

C. Ejercicio 2: La Runa Olvidada (*serialVersionUID*)

- ✓ **Directorio:** ex2/
- ✓ **Archivo:** RuneStone.java, RuneMismatch.java
- ✓ **Método:** Any.
- ✓ **Funciones Autorizadas:** serialVersionUID, InvalidClassException
- ✓ **Restricción**
 - Debes declarar explícitamente el atributo: ***private static final long serialVersionUID = 1L;***

Objetivo

Las reliquias de Asgard pueden permanecer selladas durante siglos. Pero ***¿qué ocurre si, mientras permanecen almacenadas, alguien modifica la clase que las representa?*** En este ejercicio descubrirás el papel de ***serialVersionUID***, el identificador que Java utiliza para comprobar si un objeto serializado sigue siendo compatible con la versión actual de su clase.

La misión consta de dos partes:

1. El ejercicio trata de 3 diferentes casos, lo cual no es necesario que se cumpla la salida, solo es de ejemplo, la idea principal es la 3 formas de hacerlo, tendrás que modificar RuneStone y RuneMismatch varias veces para poder realizar el problema, solo dejar la ultima manera.
2. **Serializa** un objeto **RuneStone**.
3. **Modifica la clase** (por ejemplo, añadiendo un nuevo atributo) e intenta **deserializar** el archivo generado **sin haber declarado serialVersionUID**.

Observa el resultado y documenta el error obtenido.

Después, declara un ***serialVersionUID*** fijo y rep

El programa debe:

- Crear una clase **RuneStone** serializable
 - Antes: ***value*** -> Después: ***value, name***
- Serializar una instancia de la clase.
- Modificar posteriormente la definición de **RuneStone**.
- Intentar deserializar el archivo antiguo.
- Documentar el error ***InvalidClassException*** obtenido.
- Declarar un ***serialVersionUID*** explícito.
- Repetir la prueba para comprobar la compatibilidad entre versiones.

Salida Esperada

```
PS C:\Users\Sergio URJC\Documents\java_modules\src\ex2>
java .\RuneMismatch.java
RuneStone{value=20, name='null'}
```


D. Ejercicio 3: Reliquias en Cadena (Lista Serializable)

- ✓ **Directorio:** ex3/
- ✓ **Archivo:** TreasuryVault.java, AsgardianRelic.java (name)
- ✓ **Método:** Any
- ✓ **Funciones Autorizadas:** ArrayList<AsgardianRelic>, ObjectOutputStream, ObjectInputStream, @SuppressWarnings("unchecked"), printStackTrace(), try/catch, size
- ✓ **Restricción:** No debes serializar las reliquias una por una. Debes serializar la **lista completa** con una única llamada a **writeObject()** y recuperarla posteriormente con una única llamada a **readObject()**.

Objetivo

Odin desea sellar el tesoro de Asgard en un único pergamino mágico (**treasury.ser**). Implementa una clase **TreasuryVault** que sea capaz de:

- Guardar una colección completa de reliquias en un archivo.
 - Data: Mjolnir, Gungnir, Tesseract
- Recuperar toda la colección con una única lectura.
- Mostrar cuántas reliquias fueron almacenadas y cuáles se recuperaron.

Salida Esperada

```
Treasury sealed with 3 relics.  
Treasury recovered: [Mjolnir, Gungnir, Tesseract]
```

E. Ejercicio 4: El Traidor (transient)

- ✓ **Directorio:** ex4/
- ✓ **Archivo:** SecretRelic.java, MainTest.java
- ✓ **Método:** Any
- ✓ **Funciones Autorizadas:** transient, java.io.Serializable, ObjectOutputStream, FileOutputStream, try/catch, printStackTrace, IOException, ClassNotFoundException,
- ✓ **Restricción:**
 - Al menos uno de los atributos de la clase debe declararse con la palabra clave ***transient***.

Objetivo– No toda la información de una reliquia debe almacenarse para siempre. Algunas contienen datos confidenciales, como un código de activación, una contraseña o una clave de acceso, que nunca deberían quedar guardados en un archivo serializado.

En este ejercicio descubrirás el propósito de la palabra clave transient.

Crea una clase **SecretRelic** que implemente **Serializable** y añada un atributo llamado **activationCode**. Este atributo representará un dato sensible y **no debe formar parte del proceso de serialización**. Serializa el objeto y, a continuación, deserialízalo para comprobar que el valor de **activationCode** **no se ha conservado**.

El programa debe:

- Crear una clase **SecretRelic** serializable.
- Declarar al menos un atributo con la palabra clave ***transient***.
- Asignar un valor al atributo antes de serializar el objeto.
- Serializar y deserializar la reliquia.
- Mostrar el valor del atributo antes y después del proceso.

Salida Esperada

```
--- BEFORE SERIALIZATION ---
Activation Code: THOR-7749
[Object serialized successfully...]
--- AFTER DESERIALIZATION ---
Activation Code: null
```

F. Ejercicio 5: El Pergamino Universal (JSON equivalente)

- ✓ **Directorio:** ex5/
- ✓ **Archivo:** AsgardianTreaty.java , Item.java
- ✓ **Método:** Any
- ✓ **Funciones Autorizadas:** Jackson (ObjectMapper), record, ObjectMapper, objectMapper.enable(SerializationFeature.*INDENT_OUTPUT*), printStackTrace
- ✓ **Restricción:**
 - Debes utilizar el mismo modelo de datos (**AsgardianRelic** o uno equivalente) que empleaste en el ejercicio 3, pero serializarlo en formato **JSON** en lugar de binario.

Objetivo

Hasta ahora has almacenado las reliquias de Asgard utilizando la serialización binaria de Java. Ese formato es eficiente, pero solo puede interpretarse fácilmente desde Java. En este ejercicio darás un paso hacia un formato universal: **JSON**.

Utiliza **Jackson** para convertir la misma colección de reliquias del ejercicio 3 en un archivo llamado: **treasury.json**

El archivo debe generarse con **pretty print**, de forma que resulte legible para cualquier persona o aplicación.

El programa debe:

- Crear una lista de reliquias (**AsgardianRelic** o equivalente).
- Utilizar **ObjectMapper** de Jackson.
- Guardar la lista en **treasury.json**.
- Generar el archivo con formato legible (*pretty print*).
- No utilizar la serialización binaria (**ObjectOutputStream**) en este ejercicio.

Data: "name": "Mjolnir", "powerLevel": 9000, "name": "Gungnir", "powerLevel": 8500

Salida Esperada

```
JSON file generated successfully with pretty print.
```

G. Ejercicio 6: El Veredicto de Heimdall (comparativa)

- ✓ **Directorio:** ex6/
- ✓ **Archivo:** FormatComparator.java, treasury.ser, treasury.json
- ✓ **Método:** Any
- ✓ **Funciones Autorizadas:** Files.size(Path), System.out.println
- ✓ **Restricción:**
 - La comparación debe basarse en datos objetivos. No basta con una opinión: debes medir el tamaño real de ambos archivos utilizando **Files.size()**.

Objetivo

Has almacenado las mismas reliquias de Asgard utilizando dos formatos distintos:

- **Serialización binaria de Java** (treasury.ser)
- **JSON** (treasury.json)

Ahora debes comparar ambos resultados y decidir cuál es más adecuado según el contexto. Para ello, obtén el tamaño real de cada archivo y analiza sus diferencias desde un punto de vista práctico. Para resolver este ejercicio, **no necesitas crear nuevos atributos ni nuevas clases de datos**, ya que vas a comparar los archivos que **ya generaste** en los ejercicios anteriores:

1. treasury.ser (generado con la serialización binaria en el ejercicio 3).
2. treasury.json (generado con Jackson en el ejercicio 5).

El programa debe:

- Obtener el tamaño de **treasury.ser** mediante **Files.size()**.
- Obtener el tamaño de **treasury.json** mediante **Files.size()**.
- Mostrar ambos tamaños en bytes.
- Comparar la legibilidad de cada formato.
- Reflexionar sobre cuál sería más adecuado para intercambiar datos con aplicaciones escritas en otros lenguajes.
- Escribir una conclusión final en un comentario **Javadoc** al final del archivo.

Salida Esperada

```
treasury.ser -> 165 bytes (binary, unreadable outside Java)
treasury.json -> 106 bytes (plain text, readable anywhere)
```

H. Ejercicio 7: La Línea de Sucesión (herencia + serialización)

- ✓ **Directorio:** ex7/
- ✓ **Archivo:** AsgardianRelic.java, LegendaryRelic.java, SuccessionTest.java
- ✓ **Método:** Any
- ✓ **Funciones Autorizadas:** extends, Serializable, super()

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
```

- ✓ **Restricción:**
 - Solo la clase **LegendaryRelic** debe implementar explícitamente Serializable. La clase **AsgardianRelic** no debe implementar esta interfaz.

Objetivo

No todas las reliquias nacen iguales. Algunas heredan las propiedades de antiguas armas forjadas por los dioses, mientras que otras añaden habilidades propias. En este ejercicio descubrirás cómo interactúan la **herencia** y la **serialización** cuando la clase padre **no** es serializable.

Crea una jerarquía de clases donde:

- **AsgardianRelic** sea la clase base y **no implemente** Serializable.
- **LegendaryRelic** extienda de **AsgardianRelic** e **implemente** Serializable.
- La clase padre debe disponer de un constructor que imprima: Relic forged.

Serializa un objeto **LegendaryRelic** y, posteriormente, deserialízalo. Observa cuidadosamente la salida del programa y documenta, mediante un comentario, por qué el mensaje del constructor de la clase padre vuelve a aparecer durante la deserialización.

El programa debe:

- Crear una clase **AsgardianRelic** que **no** implemente Serializable.
- Crear una clase **LegendaryRelic** que extienda de **AsgardianRelic** e implemente Serializable. Utilizar **super()** para inicializar la clase padre.
- Serializar un objeto **LegendaryRelic**, deserializar el mismo objeto.
- Documentar el comportamiento observado durante la deserialización.

Salida Esperada

```
Relic forged.
Relic sealed.
Relic forged.
Relic recovered: Gungnir
```

Reto. "El Vigía del Bifröst"

- **Directorio:** *reto/*, solo un archivo.
- **Dónde ejecutarlo:** en el mismo proyecto *hola-bosque* que ya tienes, no crees uno nuevo.

Objetivo: en este módulo aprendiste que la serialización binaria es rápida pero cerrada, y JSON es universal pero más verboso. Ahora vas a exponer esa decisión a través del navegador: un **endpoint que guarda un registro de "avistamiento del Bifröst"** y lo devuelve en formato JSON, aunque internamente lo guarde también en binario para comparar.

Qué vas a agregar dentro de tu clase `HolaBosqueApplication.java`, justo debajo del método `areaConSensor()` (del Reto 03):

Primero, una clase simple:

```
import java.io.Serializable;

static class BifrostSighting implements Serializable {
    private static final long serialVersionUID = 1L;
    private String location;
    private int intensity;

    public BifrostSighting(String location, int intensity) {
        this.location = location;
        this.intensity = intensity;
    }

    public String toJson() {
        return "{\"location\": \"" + location + "\", \"intensity\": " + intensity + "}";
    }
}
```

Y el nuevo método:

Primero, una clase simple:

```
@GetMapping("/bifrost")
public String bifrostSighting(@RequestParam String location, @RequestParam int intensity) {
    try {
        if (intensity <= 0) {
            throw new IllegalArgumentException("La intensidad debe ser positiva.");
        }
        BifrostSighting sighting = new BifrostSighting(location, intensity);
        return "Avistamiento registrado: " + sighting.toJson();
    } catch (IllegalArgumentException e) {
        return "Error: " + e.getMessage();
    }
}
```

No necesitas import nuevo salvo `java.io.Serializable`, que ya está incluido arriba.

Cómo probarlo

1. Guarda, reinicia el servidor.
2. Ve a esta URL: `http://localhost:8080/bifrost?location=Asgard&intensity=9500`
3. Deberías ver: Avistamiento registrado: {"location": "Asgard", "intensity": 9500}
4. Prueba con intensidad negativa para confirmar que el try/catch sigue protegiendo el endpoint.

Qué está pasando (sin entrar en detalle todavía)

- Aquí construiste el JSON "a mano" con un **`toString()`**-como-JSON, solo para que veas la diferencia de esfuerzo frente a usar Jackson (como hiciste en el módulo 04) — es un buen momento para notar por qué las librerías existen: hacerlo a mano es tedioso y propenso a errores si el texto tiene comillas o caracteres especiales.
- La clase implementa **`Serializable`** aunque en este **`endpoint`** no la serialices — es para que la tengas lista si más adelante quieres guardar el histórico de avistamientos en un `.ser`.

Ejecución – Si no te funciona el botón de IntelliJ

```
cd "C:\Users\XXXX\Documents\hola-bosque\hola-bosque"  
.\mvnw.cmd spring-boot:run
```

Ejecución

El flujo real en tu terminal

Entra en la carpeta:

Primero tienes que estar donde está el archivo. Si no, el comando `javac` te dirá "**archivo no encontrado**" y te entrarán ganas de tirar el PC por la ventana.

- ✓ `cd Desktop/proyectos_java/ex00`
- ✓ `javac FtHelloForest.java` -> La Compilación (El traductor)
- ✓ `java FtHelloForest` -> La Ejecución (El lanzamiento)